2024 IEEE 31st International Conference on High Performance Computing, Data and Analytics Workshop (HiPCW) | 979-8-3315-0911-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/HiPCW63042.2024.00021

Predictive Modeling of Performance Variability in HPC Applications

Pratham Sahu* (B.Tech Student), Preeti Malakar†
Department of Computer Science and Engineering, *Indian Institute of Technology Kanpur*,
spratham@cse.iitk.ac.in*, pmalakar@cse.iitk.ac.in†

***Abstract*—Performance variability is common in HPC applications, due to factors such as job scheduling, resource contention, and network traffic. We propose a machine learning-based model to predict performance variability in high performance computing applications. Our model integrates classification and regression techniques to identify jobs with high variability and accurately predict their performance fluctuations. The model incorporates job-specific and system-wide factors such as network counters to enhance prediction accuracy.**

I. Introduction

Performance variability is unavoidable in parallel programs running on large-scale clusters/supercomputers. The source of this variability may be due to myriad factors such as process placement, the parallel algorithms, the number of processes, job interference due to network congestion and resource contention, asynchronous communications, communication-computation overlap and many more [1]–[4]. Figure 1 shows the variability over 50 runs of the same configuration of the MiniMD application on 4 nodes (48 processes per node) of PARAM Sanganak supercomputer at IIT Kanpur. These were run over a period of 30 days. The variability in the runtimes can be attributed to various factors mentioned above. In this work we model the variability of a job based on the job parameters and the current state of the system, for example the number of co-scheduled jobs that may interfere with the given job. The model (explained in Section III) is built in two stages. In the first stage, we classify the jobs into two categories: jobs with high variability and jobs with near zero variability. In the second stage, we predict the variability of the jobs in the high variability category. The classification model gives an accuracy of 84.78% and the regression model gives a MAPE of 9.33%.

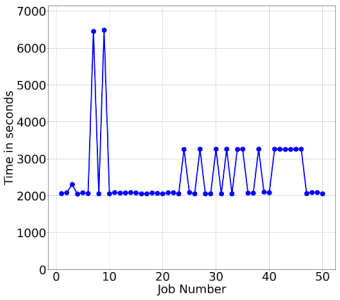

Fig. 1: Variability of runtimes for 50 runs of MiniMD application on 4 nodes (192 processes) of PARAM Sanganak supercomputer.

II. Related Work

Performance variability in HPC systems has been extensively studied. Skinner and Kramer [2] describe how system architecture and workload characteristics contribute to fluctuations in runtime. Bhatele et al. [3] showed that network topology and routing algorithms significantly impact performance variability, particularly dragonfly-based systems with high inter-node communication. Sun et al. [1] leveraged machine learning to model HPC application variability, identifying important features such as resource utilization and job scheduling patterns that affect performance. Larson et al. [5] performed large-scale monitoring on the Lassen supercomputer, providing insights into how real-time system monitoring can track and mitigate variability. Ferreira et al. [6] proposed a lightweight approach to measure variability, minimizing overhead associated with detailed performance monitoring. Although these studies offer valuable insights, our work expands on them by incorporating finer job-level and system-level details to understand and predict performance variability in HPC workloads.

III. Performance Variability Prediction

We present a machine learning based model to predict the performance variability of HPC applications using application-specific parameters such as number of nodes and system-specific parameters such as network counters. Our model combines classification and regression techniques to predict performance variability. The model leverages supercomputer job logs, network performance data, communication path information and system state (e.g. number of running jobs, jobs sharing communication paths) to identify factors contributing to performance deviations. We first used an XGBoost classifier to filter out low-variability jobs followed by Random Forest regressor to predict performance variability. Using this approach, the model can better understand and predict job performance variability. We next describe our data collection mechanism, followed by feature selection and model training.

A. Data Collection

We used job logs and system data from the PARAM Sanganak supercomputer at IIT Kanpur. It comprises 312 Intel Cascade Lake 48-core nodes connected by the Infiniband network in a two-level fat-tree topology (L1 leaf switches and L2 spine switches). It employs the SLURM scheduler.

Job Log Data: We collected and processed detailed SLURM job logs. The job log was curated from all user jobs for a period of twelve months (June 2023 – May 2024). The job log includes features such as userID, jobID, jobName, submission time, start and end times, elapsed time, number of nodes, CPUs,

2770-0135/24/$31.00 ©2024 IEEE
DOI 10.1109/HiPCW63042.2024.00021

and node list. We filtered out two kinds of jobs from this job log – (1) single-node jobs (they have lower variability) and (2) jobs exceeding their requested wall time (they could introduce noise into the analysis). Similar jobs to get deviance were grouped based on the job name, user name, and node/core count to get the average runtime for each group.

Network Data: We collected network traffic metrics such as the number of received and transmitted bytes at the compute nodes and the network switches. We collected the network data at a frequency of 10 minutes at the global system level across every device using the InfiniBand Performance Monitoring tool `perfquery` as used by Sharma et al. [7]. We aggregated and normalized these network counters to create per-job and per-path metrics, which were crucial for attributing network contention and congestion effects to individual jobs. This data shows the transmitted and received bytes at every device in a communication path, henceforth we refer to this as *path element*. An example path is $P1 : CN1 \rightarrow L1SW1 \rightarrow L2SW3 \rightarrow L1SW2 \rightarrow CN2$. $P1$ details the route of communication path between compute nodes $CN1$ and $CN2$ through the various switches , as obtained from tool. We constructed such exact communication paths (all-to-all) using the nodes allocated to a job and the network data that was collected. These paths detail the network switches and links traversed by inter-node messages during job execution. Each path entry includes a timestamp and the communication path details as shown. This data is crucial for understanding the network-level factors influencing job performance variability. This data was used to attribute network performance metrics to specific jobs and to identify sources of inter-job interference.

B. Feature Engineering

In this section, we briefly describe the data preparation stage for the machine learning phase. We created a one-hot encoding for each unique path element to represent the network communication paths in each job. This encoding enabled efficient inclusion of path information as features in the machine learning model. We extracted features such as the hour of submission and day of the week from the job submission time to capture temporal patterns in job scheduling and execution. We computed the following metrics for each job:

- maximum and average communication path length (among all the communication paths in a job)
- maximum and average transmitted and received bytes (from network counter data for a job)
- standard deviation of network counters of the devices involved in communications for a job
- L1 and L2 level aggregated counter information

These metrics provide a quantitative measure of the network's impact on job performance variability. We label-encoded to convert categorical features such as job names, user names, and node lists to convert them into numerical format suitable for machine learning models. Our output or target variable is *deviance*. This represents the relative difference between a job's runtime and the expected average runtime (as calculated for the same application runs). This metric helps capture performance variability and enables the model to predict deviations from expected job runtimes.

IV. Classifier and Regressor Training

We now present the model details along with training and test dataset. Our data being tabular, we used classical machine learning models such as XGBoost and Random Forest for prediction rather than deep learning models. The final feature set consists of 20 features which included both job-level features (e.g., number of CPUs, nodes, job name, user) and network-level features (e.g., path element encodings, transmitted and received bytes) as explained in Section III. The predictive model was developed using a combination of classification and regression techniques to accurately forecast performance variability. We explain our approach next.

A. Handling Data Skewness

The deviance values in the dataset were skewed. Many jobs exhibited near-zero variability. Thus it was essential to address this imbalance to enhance the model's predictive capabilities. We employed a two-tiered modeling strategy as explained next:

- Classification: An XGBoost classifier was trained to distinguish between jobs with negligible variability and those with significant variability (two classes). We filtered out the majority class (near-zero variability) by setting a probability threshold, ensuring that the regression model focuses on the more critical data points.
- Regression: A Random Forest regressor was subsequently trained on the subset of data with non-zero variability as identified by the classification model. This approach allowed the regression model to accurately predict the extent of variability where it was most impactful.

This methodology not only addressed the class imbalance but also significantly improved the overall performance and predictive accuracy of the models.

B. Training Process

The dataset was split into training and testing sets using an $80 : 20$ split to evaluate model performance on unseen data. There were $2,131$ unique jobs, with $1,705$ jobs used for training and 426 jobs for testing. We tuned the hyperparameters using GridSearchCV to optimize XGBoost classifier's parameters, such as the number of estimators (`n_estimators`), learning rate (`learning_rate`), maximum tree depth (`max_depth`), subsample ratio (`subsample`), column sampling by tree (`colsample_bytree`), and regularization term (`gamma`). This step helps find the best combination of hyperparameters to enhance the classification performance.

The workflow involved a hybrid approach – the outputs of two models are combined to predict the target variable, deviance. For a set of jobs J whose jobnames, number of nodes and cores, and requested time are same, the deviance of a job j is defined as the absolute difference between the actual runtime of job j and the average runtime of all jobs in J. We used XGBoost classifier to predict the probability of a non-zero deviance, while the Random Forest regressor

predicts the deviance (variability). If the classifier predicts a non-zero probability, the regressor's prediction is used; otherwise a deviance of zero is assumed. This combination of classification and regression models was designed to handle the skewed nature of the dataset where the target variable may be zero often. This improves the overall prediction accuracy. We explored other combination of models such as LGBM, CatBoost, XGBoost and Random Forest. However, the combination of XGBoost Classifier and Random Forest Regressor produced the best results. Deep learning methods were also explored, however due to tabular nature of the data, native methods performed better.

V. Results

We show the evaluation metrics, classifier threshold tuning, and feature importance results in this section.

A. Performance Evaluation

We evaluated using several metrics as listed below. These offer insights into the prediction of performance variability in HPC workloads. All of the metrics are calculated with respect to the target metric `deviance` as explained in Section IV-B.

- Mean Absolute Error (MAE): Represents the average of the absolute differences between predicted and actual values of variability. Here y_i is the actual value and $\hat{y_i}$ is the average value of all jobs in the same group.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y_i}|$$

- Root Mean Squared Error (RMSE): The square root of the average of the squared differences between predicted and actual values of variability.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y_i})^2}$$

- R-squared (R^2): represents the goodness of fit.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y_i})^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- Mean Absolute Percentage Error (MAPE): The percentage error between predicted and actual values of variability.
- Classification Accuracy: Represents the proportion of jobs correctly classified as having near-zero or non-zero variability. TP, TN, FP, and FN denote true positive, true negative, false positive, false negative respectively.

$$Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$$

Table I shows the values of the evaluation metrics explained above from our tests. MAPE can be problematic when actual values are zero, so we calculated two results for MAPE (1) We added a small constant 0.001 to avoid division by zero as shown in the table. (2) We calculated MAPE for the cases were actual values were non-zero. The first case yielded a slightly lower MAPE of 9.34% compared to 10.32% in the second case. The model demonstrates its robustness with a low MAE of 0.04, highlighting its ability to make precise predictions. If we skip the two-step approach and rely solely on regression, we achieve a significantly higher MAPE of 29.69% on the same dataset with the 80:20 train-test split. The two-step approach filters out 23% jobs as near-zero variability, out of which 85% are correctly classified. We also observe higher MAPE for lesser training data (second row).

TABLE I: Final Model Performance Results

Train:test	MAE	RMSE	R^2	MAPE (%)	Classification Accuracy
80:20	0.04	0.06	0.63	10.32	0.85
70:30	0.04	0.07	0.51	17.40	0.81

Figure 2 shows the pairplot of the actual versus predicted deviance values. Each point represents a job, with the diagonal line indicating perfect predictions. The top-left histogram displays the distribution of actual deviances, showing a concentration at lower values. The top-right scatter plot compares actual to predicted deviances, with points deviating from the diagonal indicating prediction errors, especially for smaller values. The bottom-left plot, a mirror image of the top-right, emphasizes these inaccuracies with axes swapped. Finally, the bottom-right histogram of predicted deviances similarly shows a skew towards lower values. The model captures higher performance deviations well, but smaller deviations show more dispersion, indicating some classification errors prediction errors. The model appears to capture most of the variability in jobs with higher performance deviations.

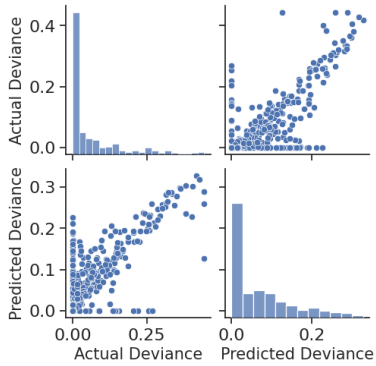

Fig. 2: Pairplot of Actual and Predicted Deviance Values

Figure 3 shows the confusion matrix for the classification model. We note that the model performs well in classifying jobs with near-zero variability, with an accuracy of 84.78%. False negatives are high, indicating that the model is more likely to predict non-zero variability for a job when it actually has near-zero variability. However, our regression model takes care of this by predicting near-zero deviance for such jobs.

B. Threshold Tuning

The classification model filtered out jobs with near-zero variability. We define threshold as the probability given by the classification model, above which we predict a value (runtime) as having zero deviance. We varied the threshold between 0.00 and 1, with the performance metrics evaluated at each step. Figure 4 shows the impact of varying the threshold on performance metrics. The plot shows the various evaluation metrics with increasing threshold values (x-axis). As observed,

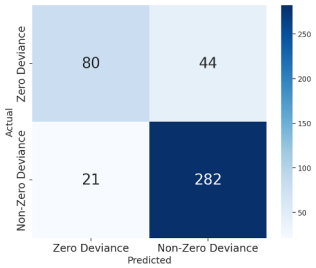

Fig. 3: Confusion Matrix for Classification Model

a threshold of 0.50 provides the best balance between regression performance (MAPE in red) and classification accuracy. This threshold effectively filtered out low-variability cases while improving model focus on more meaningful data points.

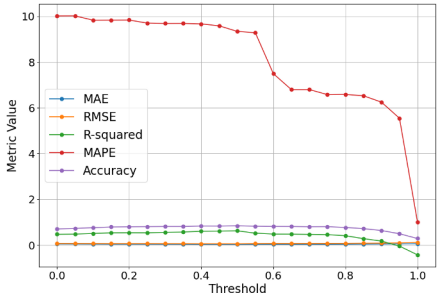

Fig. 4: Threshold Tuning Results

C. Feature Importance

Classification Model: The feature importance plot for the classification model is shown in Figure 5. The classification model's most important features include the time requested, average elapsed time, user name, number of nodes, and number of cores. This is followed by the allocated nodes (NodeList-Bitmap) and the communication path lengths (AvgPathLength). Smaller jobs typically show near-zero deviances in the dataset and hence we observe the higher correlation of deviance with time and average elapsed times.

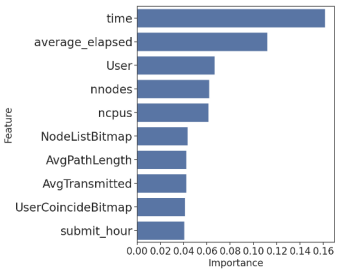

Fig. 5: Feature Importance for Classification Model

Figure 6 shows the feature importance plot for the regression model. The most important features are average elapsed time, L1 switch traffic, average network traffic received at each element of the path, L2 switch traffic, and average network traffic transmitted at each element of the path. This suggests that longer-running jobs with higher network traffic tend to have higher variability. We also observed higher traffic at the L2 switches (average 3.9×10^8) than the L1 switches (average 4.8×10^7). The average number of nodes per job was 3.34 and the count varied from 2 to 20. The average number of cores per job was 159 and the count varied from 4 to 960.

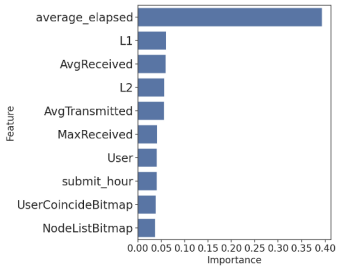

Fig. 6: Feature Importance for Regression Model

VI. Conclusions and Future Work

Our model can predict the performance variability of the jobs within a reasonable error margin given the system state i.e switch and path element level traffic information, concurrent jobs, job characteristics, and a few initial runs to determine the average elapsed time. This can be used to improve scheduling and resource allocation leading to more efficient utilization of resources and reduced average job completion times. In future, we plan to consider more details such as I/O contention and hardware state (cache misses). We plan to incorporate these factors into our model to provide a more comprehensive analysis of performance variability in HPC workloads.

Acknowledgments

We would like to acknowledge Computer Center, IIT Kanpur for providing access to PARAM Sanganak and timely assistance during the data collection process, especially to Mr. Akshay Sharma, REO.

References

[1] J. Sun, G. Sun, S. Zhan, J. Zhang, and Y. Chen, "Automated Performance Modeling of HPC Applications Using Machine Learning," in *IEEE Transactions on Computers*, 2020, pp. 749–763.

[2] D. Skinner and W. Kramer, "Understanding the causes of performance variability in HPC workloads," in *Proceedings of the IEEE Workload Characterization Symposium, 2005.*, pp. 137–149.

[3] A. Bhatele and et al., "The Case of Performance Variability on Dragonfly-based Systems," in *2020 IEEE IPDPS*, pp. 896–905.

[4] T. Cornebize and A. Legrand, "Simulation-based optimization and sensibility analysis of MPI applications: Variability matters," *Journal of Parallel and Distributed Computing*, vol. 166, pp. 111–125, 2022.

[5] T. Patki and et al., "Monitoring Large Scale Supercomputers: A Case Study with the Lassen Supercomputer," in *IEEE CLUSTER 2021*, pp. 468–480.

[6] J. Dominguez-Trujillo and et al., "Lightweight Measurement and Analysis of HPC Performance Variability," in *2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pp. 50–60.

[7] A. Sharma, D. Sahu, and P. Malakar, "Visual Analysis of Congestion and Interference in Supercomputers," in *2023 IEEE 30th HiPC Workshop (HiPCW) Student Research Symposium*, 2023, p. 71.